Expense Tracker: A Comparison of Python and C++17

Prateek Sahay and Venkata Sai Makani

University of the Cumberlands

MSCS632-M30: Advanced Programming Languages

Jay Thom

October 4, 2026

Expense Tracker: A Comparison of Python and C++17

Data Representation and Type Systems

Our Python and C++17 Expense Trackers implement identical commands and exchange CSV files. Repository: https://github.com/psahay101/cross-language-expense-tracker. Equivalent behavior lets us compare representation, expression, and ownership through our own code.

Python stores dictionaries documented by TypedDict; annotations are not enforced at runtime (Python Software Foundation, n.d.-c). C++ stores Expense values in std::vector with compiler-checked fields. A misspelled dictionary key may fail during execution; an unknown C++ member fails compilation. Both still need runtime validation of business rules.

Python uses datetime.date; C++ validates Gregorian dates before comparing fixed-width strings. Both parse 12.34 as 1234 integer cents. Python integers have flexible precision, while C++ uses std::int64_t. Shared limits preserve parity. The actual declarations below reveal these different representations.

Listing 1. Python dictionary type hints (python/tracker.py).

```
class Expense(TypedDict):
    id: int
    date: date
    amount_cents: int
    category: str
    description: str
```

Listing 2. C++ value representation (cpp/tracker.hpp).

```
struct Expense {
    std::int32_t id;
    std::string date;
    std::int64_t amount_cents;
    std::string category;
    std::string description;
};
```

Memory, Error Handling, and Modularity

Python separates procedural core functions from argparse and uses csv for decoding and quoting (Python Software Foundation, n.d.-a). C++ separates headers and sources and encapsulates operations in Tracker. Writing its CSV parser and calendar checks increased implementation effort under our standard-library-only constraint.

Python manages object memory automatically; context managers close files (Python Software Foundation, n.d.-b). C++ containers own values, while RAII streams and a temporary-file destructor release resources on exceptions (Stroustrup & Sutter, 2026). Manual new/delete is unnecessary. Python filtering collects references to dictionaries; C++ selection copies Expense values, favoring simple ownership.

Both boundaries convert exceptions into Error messages and exit status 2. Invalid files are rejected before mutation. Temporary-file replacement protects completed saves, and Python additionally flushes and fsyncs its stream. Neither version provides multi-writer locking or universal crash durability. These boundaries required careful agreement.

Listing 3. Actual category accumulation in Python (python/tracker.py).

```
    for expense in expenses:
        category = expense["category"]
        totals[category] = totals.get(category, 0) + expense["amount_cents"]
```

Listing 4. Checked accumulation in C++ (cpp/tracker.cpp).

```
    for (const auto& record : select(filters)) {
        auto& category_total = summary.by_category[record.category];
        category_total = checked_add(category_total, record.amount_cents);
        summary.overall = checked_add(summary.overall, record.amount_cents);
    }
```

Paradigms, Performance, and Development Experience

Python combines procedural functions with comprehensions. C++ combines a Tracker class, generic containers, loops, and a captured std::find_if lambda for deletion. These choices illustrate supported paradigms, not exclusive ones. Neither uses logic programming. Both remain single-threaded; concurrent writers would require additional coordination.

Python accumulates categories through dictionary lookup, then sorts for display. C++ std::map maintains ordered keys during accumulation. Both scan records and reload CSV for each command. Compilation offers optimization opportunities, but parsing, allocation, copying, and I/O also affect elapsed time. We did not benchmark, so test duration does not establish a speed ranking.

Python's concise syntax and libraries reduced writability costs. C++ needed more declarations and validation, but fixed field types, const references, and explicit ownership clarified interfaces. Named functions and separated responsibilities improved readability in both. Library availability and maintenance expectations therefore matter alongside language syntax.

Verification passed 55 behavioral/interoperability tests and 7 Python core tests, covering shared-file exchange, exact cents, inclusive filters, quoting, and rejection without destructive saves. This verifies the tested scenarios. Our main lesson is that equivalent behavior requires a shared specification and boundary tests while leaving room for language-specific idioms.

References

Python Software Foundation. (n.d.-a). *csv—CSV file reading and writing*. Python documentation. https://docs.python.org/3/library/csv.html

Python Software Foundation. (n.d.-b). *Design and history FAQ*. Python documentation. https://docs.python.org/3/faq/design.html

Python Software Foundation. (n.d.-c). *typing—Support for type hints*. Python documentation. https://docs.python.org/3/library/typing.html

Stroustrup, B., & Sutter, H. (Eds.). (2026, June 14). *C++ core guidelines*. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines